

Compilation du programme OCR

8 octobre 2026 — Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical

Fichiers nécessaires

Conserver ensemble `main.c`, `reconnaissance.c`, `reconnaissance.h`, `parametres_ocr.h`, la police originale `FontRasterized_0_1.h`, ainsi que `OCR.sln` et `OCR.vcxproj`. Le projet n'utilise aucune bibliothèque externe. La police est fournie par le professeur, sans modification.

Windows avec Visual Studio

1. Sur la VM du cours, ouvrir `OCR.sln` avec Visual Studio doté des outils C/C++.
2. Choisir **Release**, plateforme **x64**, puis Build > Build Solution. Le projet utilise C11, le niveau d'avertissement 4 et place `OCR.exe` dans le dossier des sources. Ses chemins sont relatifs.
3. Si l'IDE demande de cibler le SDK/outillage, choisir la version installée dans la VM. Ne pas inventer un chemin d'installation personnel dans le projet.
4. Conserver `.sln` et `.vcxproj` dans le rendu, même si la commande ci-dessous sert au test rapide.

Alternative depuis **Developer Command Prompt for Visual Studio**, dans le même dossier :

```
cl /nologo /std:c11 /W4 /TC main.c reconnaissance.c /Fe:OCR.exe
```

`_CRT_SECURE_NO_WARNINGS` est défini avant les inclusions ; `fopen` reste utilisé comme dans le cours. Il faut une version prenant en charge C11. Ce projet a été vérifié comme configuration XML et fichiers relatifs ; **aucun build MSVC ni essai Windows n'est revendiqué ici.**

GCC ou Clang

Depuis le dossier `c` :

```
cc -std=c11 -Wall -Wextra -Wpedantic -Werror -Wvla main.c reconnaissance.c -o OCR
```

Sous Windows avec GCC, remplacer `cc` par `gcc` et utiliser `-o OCR.exe`. Ne pas renommer un exécutable macOS en `.exe`. Le binaire fourni dans `bin/macos-arm64/OCR` est uniquement pour macOS Apple Silicon ; la recette Windows demande de compiler et tester `OCR.exe` dans la VM.

Arguments et résultat

Ordre obligatoire : chemin complet du fichier, seuil vide, seuil zéro, seuil un. Les seuils sont des entiers **1 à 99 inclus**. Les valeurs normales sont **88 90 90**. Les fractions, même écrites `90.0`, ne sont pas une syntaxe acceptée ; un éventuel signe+ et des zéros initiaux sont admis.

```
"chemin complet vers OCR.exe" "chemin complet vers Cell0.bin" 88 90 90
```

Sous `cmd`, lire `echo %ERRORLEVEL%` immédiatement après l'appel. Sous macOS, appeler le chemin du binaire `OCR` et lire `echo $?`. `CellValue.txt` est créé à côté du fichier d'entrée, quel que soit le dossier courant du terminal. Le score conserve six décimales et la ligne se termine par LF (ou CRLF avec le mode texte Windows).

Exemple Moodle	Code	Ligne attendue
Cell0.bin	0	d: '0', 97.167969%

Exemple Moodle	Code	Ligne attendue
Cell1.bin	0	d: '1', 97.753906%
CellEmpty.bin	0	d: '-2', 0.000000%
BadCell_1.bin	3	diagnostic de pixels manquants sur stderr

La largeur et la hauteur doivent chacune être entre 50 et 1000 inclus ; le payload contient exactement largeur×hauteur octets 0/1. Le choix d’arrêt sur excès de pixels ou absence de chiffre est documenté dans docs/Documentation_projet.md. À code non nul, ne jamais consommer un ancien CellValue.txt. À succès, stdout et stderr restent vides.

Une paire de guillemets simples ou doubles encore présente autour de argv[1] est retirée. Dans cmd, seules les doubles guillemets groupent un chemin avec espaces. Le chemin est limité à 1023 caractères après retrait de la paire ; son allocation est dynamique.

Validation réellement obtenue

Compilation stricte Apple Clang 21, macOS 27.0.1, arm64 : réussie, sans avertissement. Les bornes de dimensions, seuils, exemples et erreurs sont contrôlés par des appels au vrai exécutable. Les tests ASan/UBSan sont des outils de vérification, pas des dépendances du programme. Les résultats précis sont dans docs/recette_labview/Verification_donnee.md.

Le projet complet ne sera validé qu’après compilation Windows, construction des VIs et exécution MATLAB/LabVIEW sur la VM. Le guide contient ces étapes et l’inventaire de remise.